A Report Of Team Project: Klotski On Game Boy

Members Zewan Lyu(999019888), Zeyan YIN(999020183), Weihan Li(999019797), Xiang Li(999019771)

1 Introduction

This is a report of the team project: Klotski on game boy. This version of Klotski is designed to have eight different levels with increasing difficulty. After entering the game players will see the homepage where they can choose their desired level. In each level players will see nine blocks of different size and the biggest block that represent the main character of this game manifested by its unique pattern. The ultimate goal is to move him to the middle bottom of the chessboard so that he could leave from the entrance. This game leaves some blank space for users to showcase their ability and skills. All blocks are free to move except that any of them cannot either overlap or move onto the boarder. After reaching the goal players will return to the homepage and selecting the other level they want to challenge.

2 Keys

There are six keys to be used while playing: A, B and four arrow keys. Mention that all keys correspond to Game Boy buttons instead of the keyboard setup in computers. By default this game is played on the emulator: Emulicious. The keys are mapped as follows: S is the A button, A is the B button, and so on.

2.1 Usage of keys when playing

1. Firstly, the game is in the state of selecting. The player select a block using the arrow keys.
2. Press A to change the state into moving.
3. Press A again to finalize the movement and select another block to slide.
4. Press B to come back to the menu(You can reset the game in the menu).

2.2 Usage of keys when selecting level(in menu)

1. Select a level using arrow keys
2. Press A to load into the chosen level

3 Graphics

3.1 Block Size

In the unit of tile (8 by 8 pixels for each tile), the screen consists of 18 rows and 20 columns of tiles. If we want to implement a Klotski with 5×4 square blocks, then each square block(the smallest block) has a maximum size of $3tiles \times 3tiles$. To show the effect that one object is selected, we need to reverse the color of the selected block.

3.2 Block Update

If the block size of last subsection is implemented, then there will be 180 tiles to be handled. But the capacity of OAM is limited to 40 tiles. So one possilble solution is to modified the background in the period of VBlank.

Similar to shadowOAM, we created the buffer shadowSCRN to be modifed. And in the period of VBlank, we copy shadowSCRN to SCRN selectively to avoid being lack of time.

The big chanllege is that the whole affected region might cover the whole slot and moreover even in the case we only modify the region in the slot the VBlank is not enough.

An easy solution is to use two VBlanks to load the graphics which may cause lag. So we choose the following way.

The function described in subsection 4.6 stored in WRAM the range of affected region. We read the region and compute the height of the affected region. In VBlank, the program calls one of two different functions according their height to make use of time efficiently.

4 SlidingBlocks

4.1 State of the coordinates

We consider different kinds of blocks categoried by size. Each smallest square block is a point with integer coordinate. Each coordinate has two states, occupied or non-occupied. The player choose one of the four directions to slide. When the state of the next coordinate in that direction is occupied. The sliding is invalid. Otherwise it is valide so the program could update the position of the block and the state of the coordinates.

So we can store the initial information of a configuration in ROM. In detail, the initial information is a 7×6 matrix. You can see in the next subsection the way the configuration be encoded into the matrix.

4.2 Matrix(Configuration)

At begining, a 7×6 matrix is loaded from ROM into WRAM which indicates the initial information of the game. Each element of the matrix contains one byte. One byte contains 8 bits, we represents the highest bit as the eighth bit and the lowest bit as the first bit(And so on). The higher 4 bits reprents a number ranging from 0 to 5.

1. 0 represents that the position is not occupied.
2. 1 to 4 represent different block types.
3. 5 represents one part of the wall.

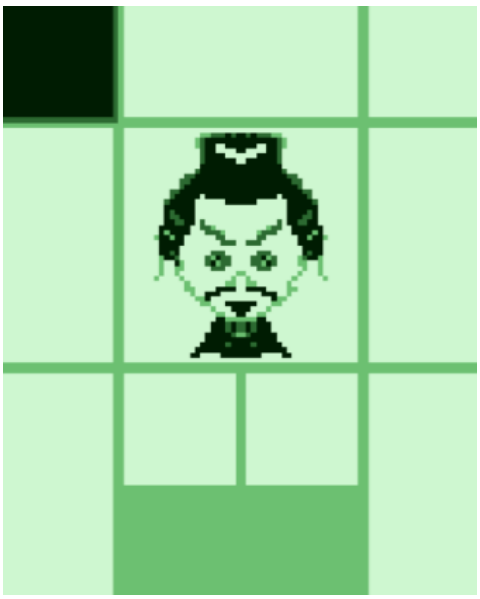
The lowest two bits of the number indicate the part of the block. For example, 0 indicates that this position is the topleft of the block. The thid bit represents whether it is selected.

In summary, the high 4 bits of each byte (bit 7-4) indicates the type of block (0-5),bit 2 indicates whether it is selected, and the low 2 bits (bit 1-0) indicates which part of the block:

- 00: Top left corner
- 01: top right corner
- 10: Lower left corner
- 11: Lower right corner

Here is one example of encoding a game level. The image in the following should be encoded into

```
SECTION "Configurations", ROM0
level1:
DB %01010000,%01010000,%01010000,%01010000,%01010000,%01010000
DB %01010000,%00010100,%00100000,%00100001,%00010000,%01010000
DB %01010000,%00110000,%01000000,%01000001,%00110000,%01010000
DB %01010000,%00110010,%01000010,%01000011,%00110010,%01010000
DB %01010000,%00110000,%00010000,%00010000,%00110000,%01010000
DB %01010000,%00110010,%00000000,%00000000,%00110010,%01010000
DB %01010000,%01010000,%01010000,%01010000,%01010000,%01010000
...
```



The size of the matrix is 7×6 while the image is composed of 5×4 smallest blocks. This is because all the blocks are surrounded by walls whose thickness is the width of the smallest block. The topleft square block is selected thus the third bit of the second byte in line4 is 1 while the third bit of any other byte is 0.

At initial, the old matrix is initialize with all the bytes being %01010000. And the new matrix is loaded from ROM. And then the program simply applies the functions in subsection 4.6 to display the loaded matrix.

4.3 SelectingBlocks

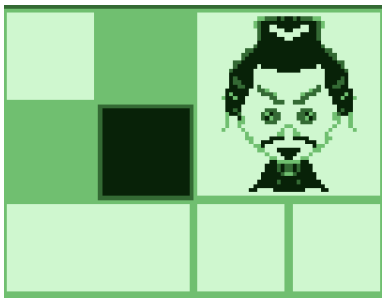
We have one currently selected block corresponding to its topleft position in the matrix. The function will change the currently selected block according to the key the player pressed.

Firstly, key A should get attention because it checks whether the the state of selecting is transited into the state of moving. A variable **stateOffSet** stores the currently game state and uses it as a condition: if it is set, then the selecting process is terminated and the moving process begins; otherwise, the selecting process carry on.

The selecting process will choose the closest block in that direction by checking the matrices.

- If a block has width or height of length one, then the program will check the block in one line.
- If a block has width or height of length two, then the program will check two lines and choose the closest one.

Now a special case appear.



In this case, the top left block can't be selected. So we implements four functions to detect four special cases.

4.4 MoveBlocks

The currently selected block corresponds to its top-left position in the matrix. The player press one of the arrow keys to select one direction, and then the sliding function will check whether the selected direction is occupied.

Firstly, the role that key A plays is the same with last subsection: it changes the game state(selecting or moving).

The detection of target movement must be implemented first in case of invalid put, for instanse overlapping, moving out of bound or lacking of sufficient space.

In general, if the move is valid, then the program first empty the block itself in newMatrix then copy the block from old matrix to the new matrix with each block shifted in the direction.

4.5 NewMatrixFormedBySelectingOrMoving

After selecting or moving blocks(last two subsections), the function will form the new Matrix.

1. The old matrix is copied to the new matrix with some elements modified to reflect selecting or moving.
2. The old matrix is copied to the new matrix(which means that the action is invalid).

4.6 MatrixToShadowSCRN

This function scans both the old Matrix and the new Matrix. Using the decoding method of 4.2, the program interprets the numbers in the matrix to determine the type of block, the specific part of the block they present and whether the current part is selected. The function uses two maps.

1. There is one map from positions of numbers in the matrix to the corresponding positions in the shadowSCRN. We could compute the corresponding positions in the shadowSCRN with respect to the positions of matrix, namely, computing the offsets respectively. As is stated in subsection 3.1, each square block has a maximum size of $3tiles \times 3tiles$ and the size of background is 18×32. So we could get the corresponding table of offsets described in SECTION "matrixToSCRNTable" in table.inc.

2. There is one map from the numbers in the matrix to the tileID matrix. Using the encoding method of 4.2, we know the specifc part of specific block the byte represent along with whether the current part is selected. So using the ByteDecoder function one can get the offsets to access the corresponding address of 3×3 tileID matrix(As is stated, the smallest block is composed of 3×3 tiles).

Finally, once the program know the address of 3×3 tileID matrix and the corresponding address of shadowSCRN it uses unitDraw method to copy the 3×3 tileID matrix to the right place of shadowSCRN.

So if there are differences between the two matrices, then the function modifies shadowSCRN respectively. And also in this procedure, the new matrix is copied to old matrix.

5 Menu

In general, we use the same data structure and functionality to display the menu as we did for the slider.

The selection box in the menu interface is also composed of 3x3 tiles, using the mouse pointer (implemented by the moving sprite) to select the level. After confirming the selection, switch to the game state using the stateOffset variable.

6 Game State Counters

The game uses two counters to track state:

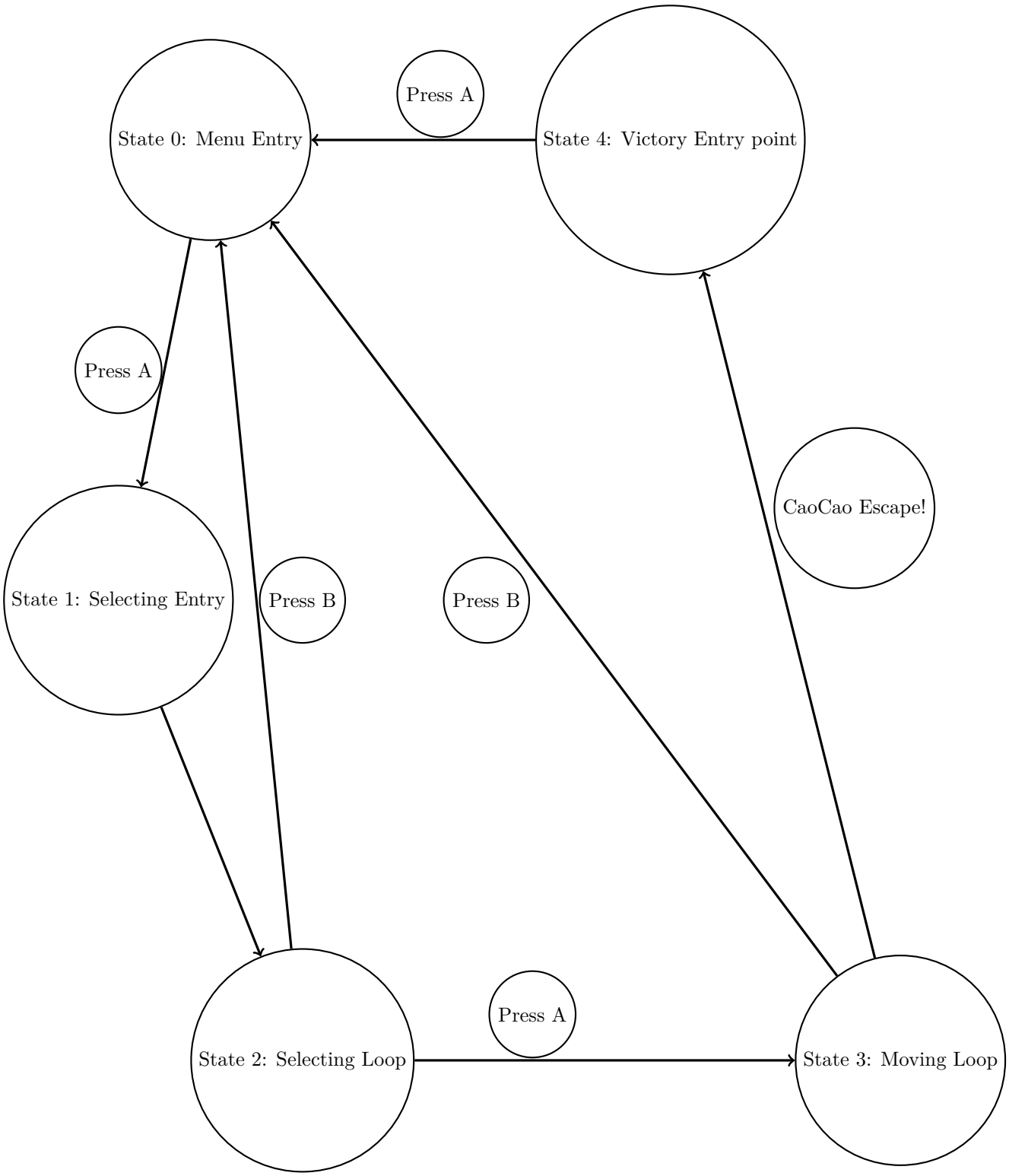
- LevelCounter: Record the current level number (1-8), displayed at the top left of the screen
- StepsCounter: Records the number of steps the player moves, displayed at the top right of the screen

These counters are stored in WRAM and updated on the screen via the stepCounter function.

7 Finite State Machine

To make the structure clearer, we apply Finite State Machine. We use a variable **stateOffset** to store the current game state.

The following image shows all states and the transitions between states.



8 Optimization

In order to complete the drawing in the limited VBlank time, we adopted the following optimization strategy:

- uses shadow SCRN buffers to reduce VRAM access
- selects a different update strategy depending on the height of the affected area
- expands the loop to reduce the expence of the function call
- updates only the area that has changed